COMPONENT BASED SOFTWARE ENGINEERING

Architecting Component-Based Systems

Introduction

- In general, software architecture serves as a blueprint, outlining how a system is structured and organized. It determines whether your system will be scalable, maintainable, and how well it will perform.
- Currently, more organizations and developers are turning to component-based architecture (CBA) to build complex software systems.
- This approach emerged to address the need for highly scalable, loosely coupled solutions. With CBA, an application is split into smaller, manageable, and reusable parts called components.
- Each component contains specific functionality and interacts with others through well-defined interfaces.

What is component-based architecture?

- Component-based architecture is a framework for building software based on reusable parts. Each component contains well-defined functionality in a binary unit that is stored in a library and then dropped into an application without modifying other components.
- When building a structure out of Legos, you can choose various shapes, sizes, and colors. Some blocks are purpose-built to be doors, windows, and other structural elements. Each block has all the features it needs to connect to others, and adding or subtracting blocks generally has minimal impact on the structure.
- Software components are much more complex than little pieces of plastic, but the concept is similar.
- Each element performs a task in an architecturally defined way. For simplicity, components are stored in a library, assembled by developers, and talk to each other through APIs.

What is Software Connector?

- software connectors are architectural building blocks that mediate, coordinate, and facilitate interaction, data exchange, and control flow between independent components.
- A connector in software architecture refers to an intermediary that facilitates communication and coordination between different software components or systems. It acts as an intermediary, facilitating the exchange of data, requests, or responses between different systems, allowing them to work together seamlessly.

Types of Software Connectors

- **Procedure Call Connectors:** is a fundamental software connector type that models the flow of control and transfers data between software components via routine or method invocations. Allow a component to directly invoke procedures, methods, or functions of another component (similar to standard programming language function calls or RPC - Remote Procedure Calls).
- **Event-Based Connectors:** Event-based connectors are communication mechanisms that allow software components to interact by generating, sending, receiving, and responding to events. An event is a notification that something has happened in a software system. In component-based architecture, one component can generate an event, while another component responds to it without necessarily knowing the internal details of the component that generated it.

Event-based connectors	Procedure-call connectors
Components communicate through events.	Components communicate by calling operations or methods.
Often support asynchronous communication.	Often involve a direct request and response.
Usually promote loose coupling.	Can create stronger dependencies between components.
Example: notifying a component that a payment succeeded.	Example: calling a method to calculate the total price.

Cont.:

- **Data-Flow Connectors:** Data-flow connectors are communication mechanisms that transfer data between software components so that the data can be processed, transformed, or stored. Unlike procedure calls that transfer control, these connectors are primarily concerned with the **flow of data** and often handle synchronization implicitly.
- **Client-Server Connectors:** serve as the architectural "glue" that separates interaction from computation, explicitly modeling the request-response communication between independent software modules. Manage request-reply protocols where a client component sends a service request and blocks or waits for a response from a server component.

Benefits of Connectors

- **Decoupling:** Connectors separate the components, allowing them to be developed, deployed, and scaled independently.
- **Flexibility:** They enable different components to interact using various communication protocols, making the system adaptable to changes.
- **Interoperability:** Connectors facilitate the integration of heterogeneous systems and technologies.
- **Scalability:** By managing communication, connectors can help balance load and distribute requests efficiently.
- **Maintainability:** Changes to the communication logic can be made independently of the components, simplifying system updates and maintenance.
- **Reliability:** Connectors can provide mechanisms for error handling, retries, and failover, enhancing the robustness of the system.

Architectural Style

- an **architectural style** dictates how independent, reusable components are structured, encapsulated, and coordinated.
- Architectural style defines how the components of a software system are organized and how they interact with one another.

Layered Architecture as an Architectural Style

- Layered architecture is one of the classic **architectural styles** It organizes a system into horizontal layers, where: Each layer has a specific responsibility (e.g., presentation, business logic, persistence).
- A layer typically only interacts with the layer directly below it (strict layering) or with any lower layer (relaxed layering).
- Layers expose well-defined interfaces and hide their internal implementation.
- This matches the definition of an architectural style: a named, reusable set of design decisions about structure and interaction.

Client-server component architecture

- **Client-server component architecture** is a distributed software design model where an application's workload is split between two primary entities: clients (which request services or resources) and servers (which provide them).
- This architecture structures application logic into three core layers—**presentation, processing, and data**—and relies on several critical components to function

Key Technical Components

- The architecture relies on several interconnected software and hardware components to execute a request-response cycle:
- **Client:** The user-facing program or device (e.g., web browsers, mobile apps) that initiates communication by sending a request over the network.
  - *Thin Client*: Only handles presentation logic; relies entirely on the server for processing.
  - *Fat/Thick Client*: Runs some processing and business logic locally before communicating with the server.
- **Server:** A high-capacity computer or background process that continuously "listens" for incoming requests, executes the requested tasks, and returns a response. A single server environment may distribute roles into specialized nodes like file servers, database servers, and mail servers.

Cont.:Key Technical Components

- **Network & Protocols:** The communication channel (including routers, switches, and firewalls). Components communicate using standardized protocols (like HTTP, HTTPS, or gRPC) to ensure they agree on data formats.
- **Database:** The centralized repository (e.g., SQL or NoSQL databases) where the application's persistent data is organized and secured.
- **Middleware / Load Balancers:** Software components that sit between the client and server to handle routing, authentication, logging, or traffic distribution across multiple server instances

Common Architectural Tiers

Tier Type	Description	Best Used For
2-Tier Architecture	The client handles the interface and business logic, communicating directly with a database server.	Simple internal tools, small database applications.
3-Tier Architecture	Introduces an application server between the client UI and the database to handle the business logic independently.	Standard web applications, e-commerce platforms.
N-Tier Architecture	Blends multiple specialized microservices, caching layers, and enterprise service buses into a deeply segmented system.	Scalable enterprise software, streaming services.

Others styles include

- **Microservices** are a software architectural style in which an application is divided into small, independent services, with each service responsible for a specific business function and communicating with other services through well-defined interfaces.
- In component-based architecture (CBA), microservices can be viewed as independently deployable software units that contain components responsible for particular tasks

Cont.: Microservices

- For example, when a customer places an order:
  - The Order Service records the order.
  - The Payment Service processes the payment.
  - The Delivery Service arranges shipment.
  - The Product Service updates stock availability.

Event-driven architectural style

- Event-driven architectural style is a software architecture in which components communicate and coordinate their activities by generating, transmitting, and responding to events.
- An event is a notification that something has happened in a system, such as a user clicking a button, a payment being completed, or a new order being placed.
- In event-driven architecture (EDA), components do not call each other directly. Instead, they **react to events**. An event is a significant change in state (e.g., "order placed", "temperature exceeded"). The flow of control is **implicit** — the publisher of an event does not know (or care) which components will react.
- Individual components can produce or respond to events to perform different tasks without requiring direct communication between every component.

Example in a mobile application

- Consider a mobile shopping application.
- When a customer successfully places an order:
  - The Order Component generates an OrderPlaced event.
  - The event broker distributes the event to interested components.
  - The Notification Component sends an order confirmation.
  - The Inventory Component updates the available stock.
  - The Delivery Component begins preparing the shipment.

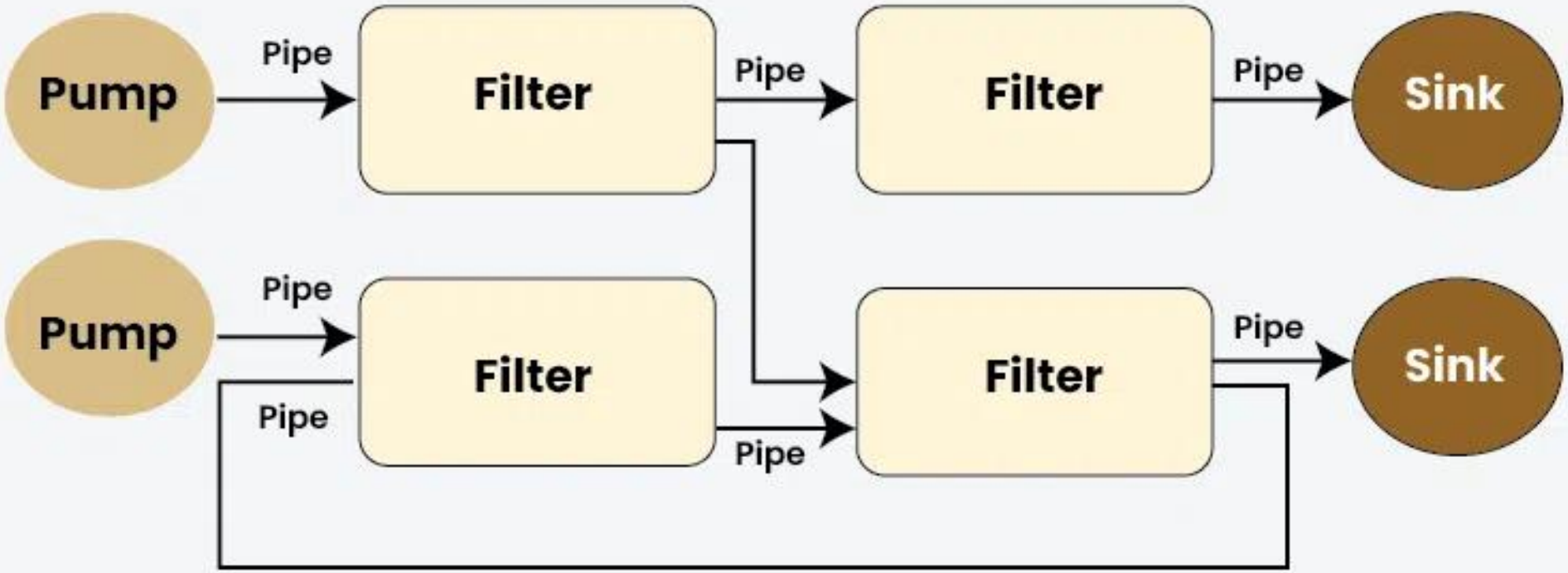
Pipe-and-filter architectural style

- Pipe-and-filter architectural style is a software architectural style in which a system is organized as a sequence of independent processing components called filters, connected by communication mechanisms called pipes.
- Each filter receives data, processes or transforms it, and passes the resulting data to the next filter through a pipe. Each filter is dedicated to a particular processing function, whether it involves transforming, validating, or aggregating data. Data flows through these filters via pipes, which carry the output of one filter to the input of another.

Cont.: Pipe-and-filter architectural style

- This architecture enhances modularity, as each filter operates independently and concentrates on a singular function. It also promotes reusability, allowing filters to be utilized across different systems or applications.
- Moreover, it is both flexible and scalable; filters can be added, removed, or rearranged with little effect on the overall system, and multiple instances of filters can function concurrently to manage larger data sets.

Pipe and Filter Architecture - System Design



Example

- Consider a mobile application that calculates students' examination results.
  - Input filter: Receives students' marks.
  - Validation filter: Checks that the marks fall within the permitted range.
  - Calculation filter: Calculates the average mark.
  - Grading filter: Assigns a grade based on the average.
  - Output filter: Displays the student's results.
- The pipes transfer the data between these filters in the required order.

Component composition

- Component composition is a design pattern where you build complex user interfaces by combining smaller, independent, and reusable components like Lego bricks. Instead of writing a single monolithic component bloated with dozens of props and conditional checks, you distribute responsibilities across cooperating parent and child component
- Component composition is the process of combining two or more software components to form a complete software system or application.
- In component-based architecture, each component performs a specific function and interacts with other components through defined interfaces.
- The aim is to assemble components so that they work together to satisfy the requirements of a software system.

Methods of component composition

1. Sequential composition: Components are connected in a sequence, where the output of one component becomes the input of another. Example: A component validates data before another component processes it.

2. Hierarchical composition: Components are organized into levels, where a higher-level component coordinates lower-level components. Example: An order management component coordinates payment and delivery components.

Cont.: Methods of component composition

3. Additive composition: Components are combined to add new functionality to an existing system. Example: Adding a notification component to an existing mobile application.

4. Parallel composition: Components perform different tasks concurrently or independently.

Example: After an order is placed, separate components process inventory updates and customer notifications.

Note: These are common ways to describe component composition; terminology and classifications may differ across course materials.

Steps involved in component composition

- Identify the system requirements.
- Identify or select suitable software components.
- Examine component interfaces and dependencies.
- Connect compatible components using appropriate connectors.
- Integrate the components into a complete system.
- Test the integrated system to ensure that the components work correctly together.

Advantages of component composition

- Reusability: Existing components can be reused in different applications.
- Reduced development time: Developers do not need to build every function from scratch.
- Maintainability: Individual components can be modified independently when interfaces remain compatible.
- Flexibility: Components can be replaced or added to extend functionality.
- Improved productivity: Development work can be divided among different teams.

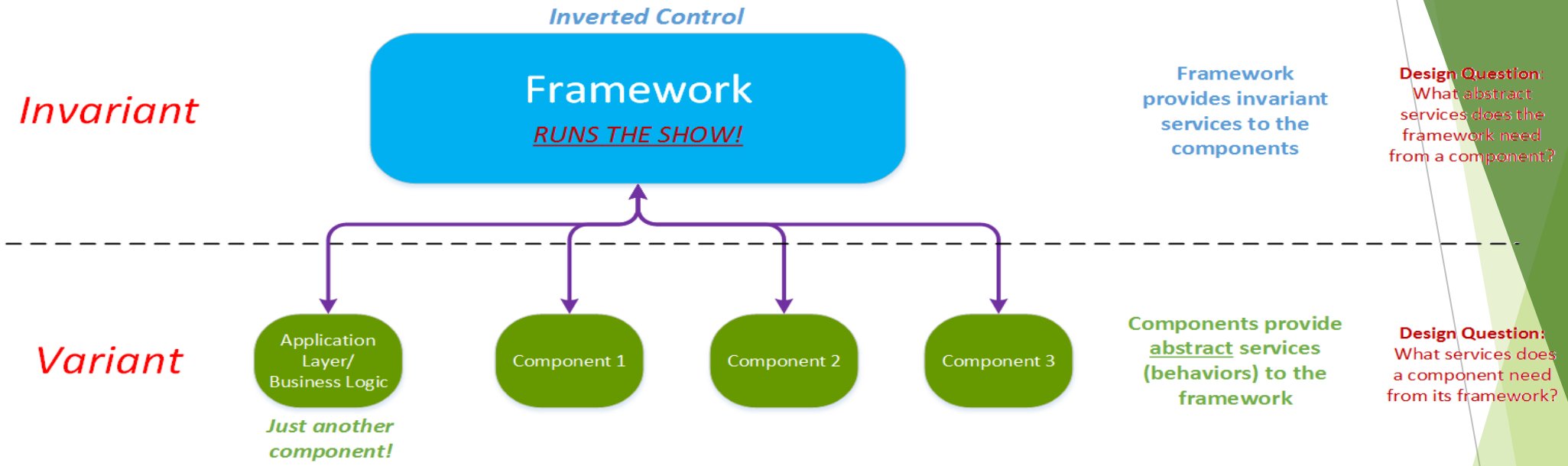
framework

- *A set of cooperating classes that makes up a reusable design for a specific class of software. A framework provides architectural guidance by partitioning the design into abstract classes and defining their responsibilities and collaborations. A developer customizes the framework to a particular application by subclassing and composing instances of framework classes.“*
- In essence, component-frameworks break the system down into variant "components" that represent abstract processing that the system supports.
- These components are generally thought of as "pluggable" in the sense that the system can be configured with a set of components to accomplish a particular task or set of tasks.
- The framework has the effect of decoupling or isolating the components from each other since it the component interactions are handled by the framework in a manner that is opaque to the components

key characteristics of a component-framework system

- *its "inversion of control" or "Hollywood Principle": "Don't call us, we'll call you!"* Since the framework portion of the system represents the invariant aspects of the system that mediates the dynamics of the variant components, *control of the system must be delegated to the framework* because the components are only providing services to the framework and are unaware of how those services interact with each other.
- The framework that is "running the show", directing the calls to the component services, not the other way around. Inversion of control virtually eliminates flow control in the components and thus reduces code complexity in the components.
- Frameworks can vary greatly in size, ranging from small data structures to enterprise-class distributed applications.

Component-Framework System



Another viewpoint:



Framework isolates components

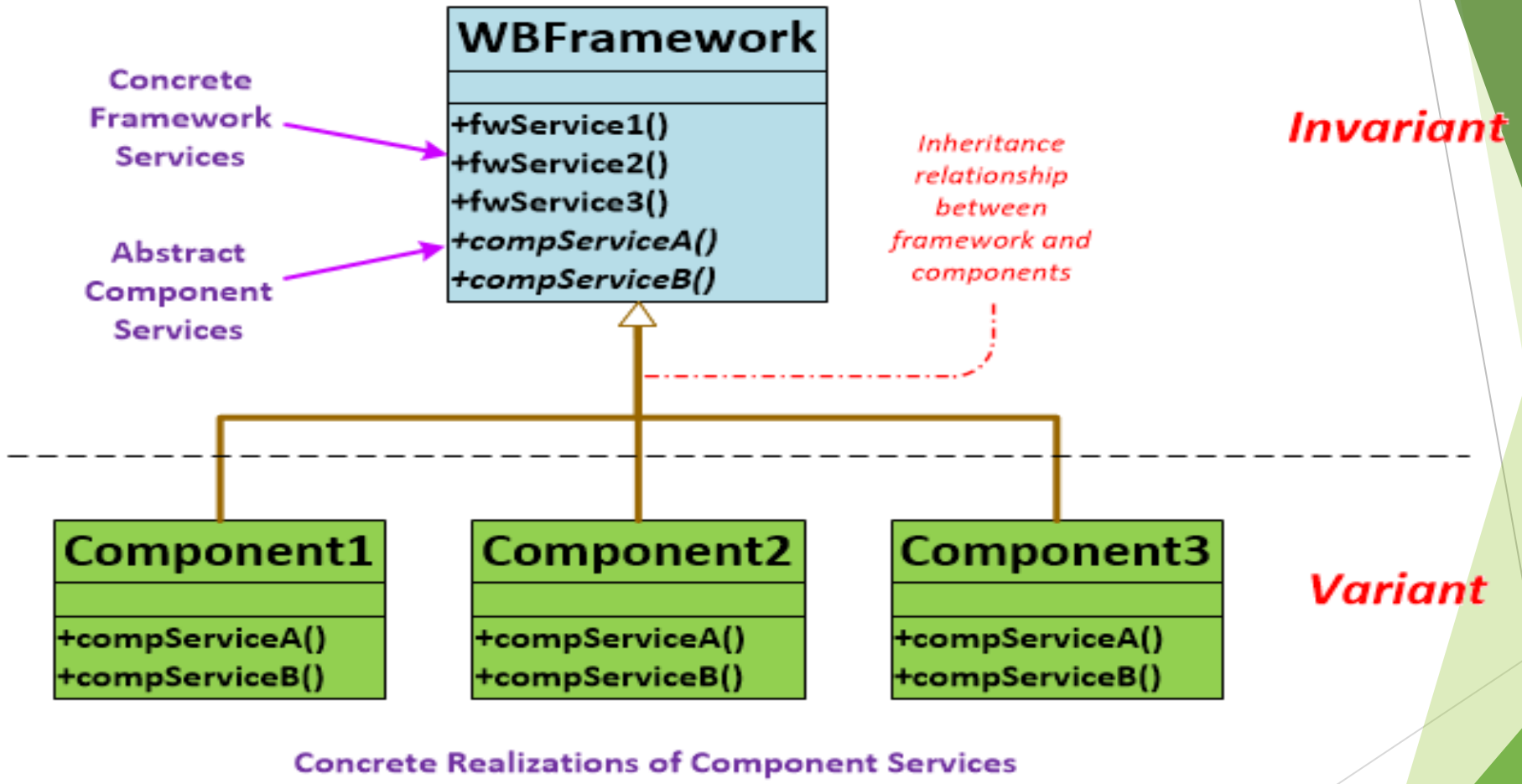
Black-box vs. White-box Frameworks

Component-framework systems come in two basic "flavors": white-box and black-box. The two types differ in how the framework relates to the components.

White-box framework

- *In a white-box framework, the components have an inheritance relationship with the framework, i.e. are subclasses of the framework.*
- That is, the framework services are concrete methods that each component inherits from their superclass(es).
- The implementation of the framework is thus in the component superclass(es).
- The framework calls to the components by calling abstract methods in the superclass from the concrete framework methods.
- The concrete component classes override these abstract methods with their specific behaviors.
- This calling of abstract methods from concrete methods is called the Template Design Pattern and is characteristic of white-box frameworks.

White-box Framework



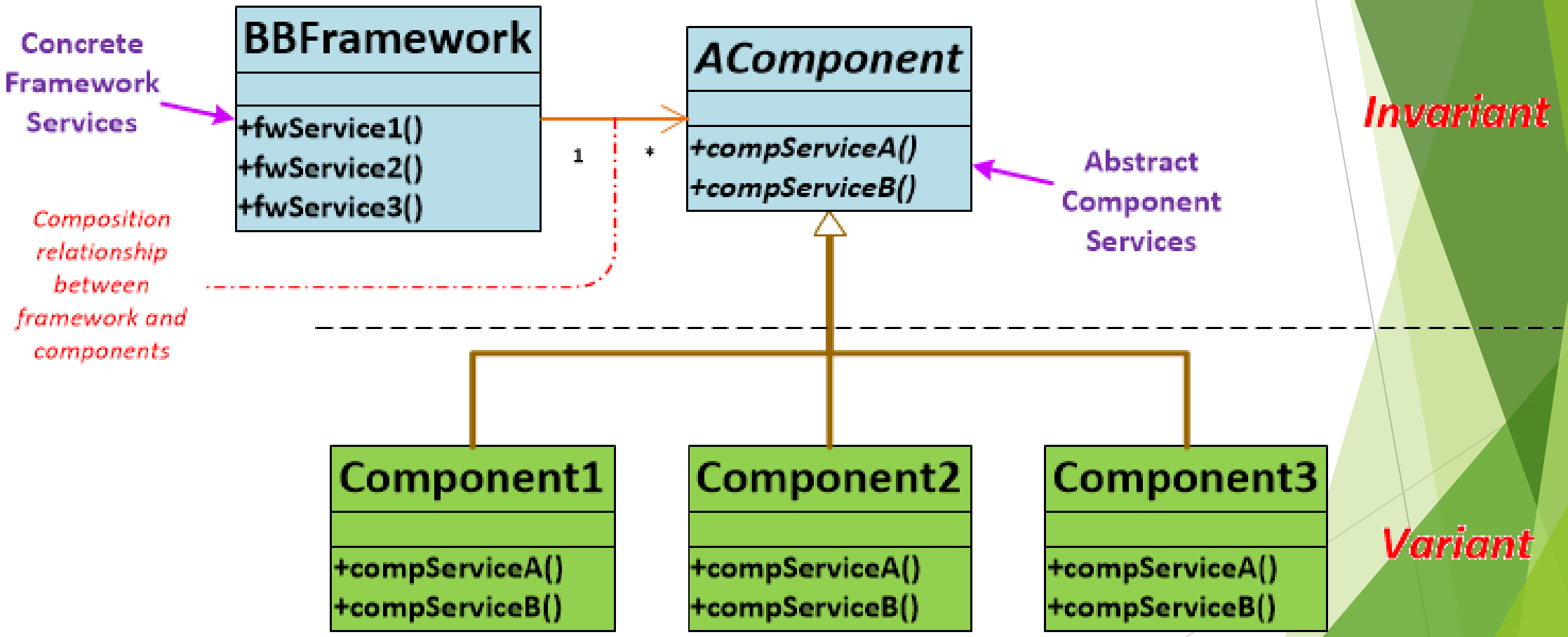
black-box framework

- *In a black-box framework, the components have a compositional relationship with the framework, i.e. are typically fields of the framework*. Here, the framework and components are completely separate objects with no inheritance relationship whatsoever.
- Typically, the compositional relationship is bi-directional so that not only can the framework call on services provided by the component, but the component in turn, can call on services provided by the framework.

Cont.: *black-box framework*

- Framework services manifest themselves as methods of the framework object while component service are methods of the component objects.
- Since the components are variant entities, the framework must reference the components as abstract entities with abstract behaviors (methods).
- This delegation by the framework to an abstract entity to fulfill some operational purpose is the Strategy Design Pattern and is characteristic of black-box frameworks.
- Often, a black-box framework is a specific, concrete entity but for systems that need upgradeability in the framework without disturbing the components, the framework may present itself to the components as an abstract entity.

Black-box Framework



Cont.:

- White-box frameworks tend to create greater coupling between the components because they are all required to have the same inheritance ancestry.
- While a disadvantage in many systems, it is advantageous in systems where tightly regulated control of the component processes is required.
- Such systems include graphical user interfaces (GUIs) and security architectures.
- White-box frameworks also tend to be easier to understand and program because of the fewer objects involved.

Cont.:

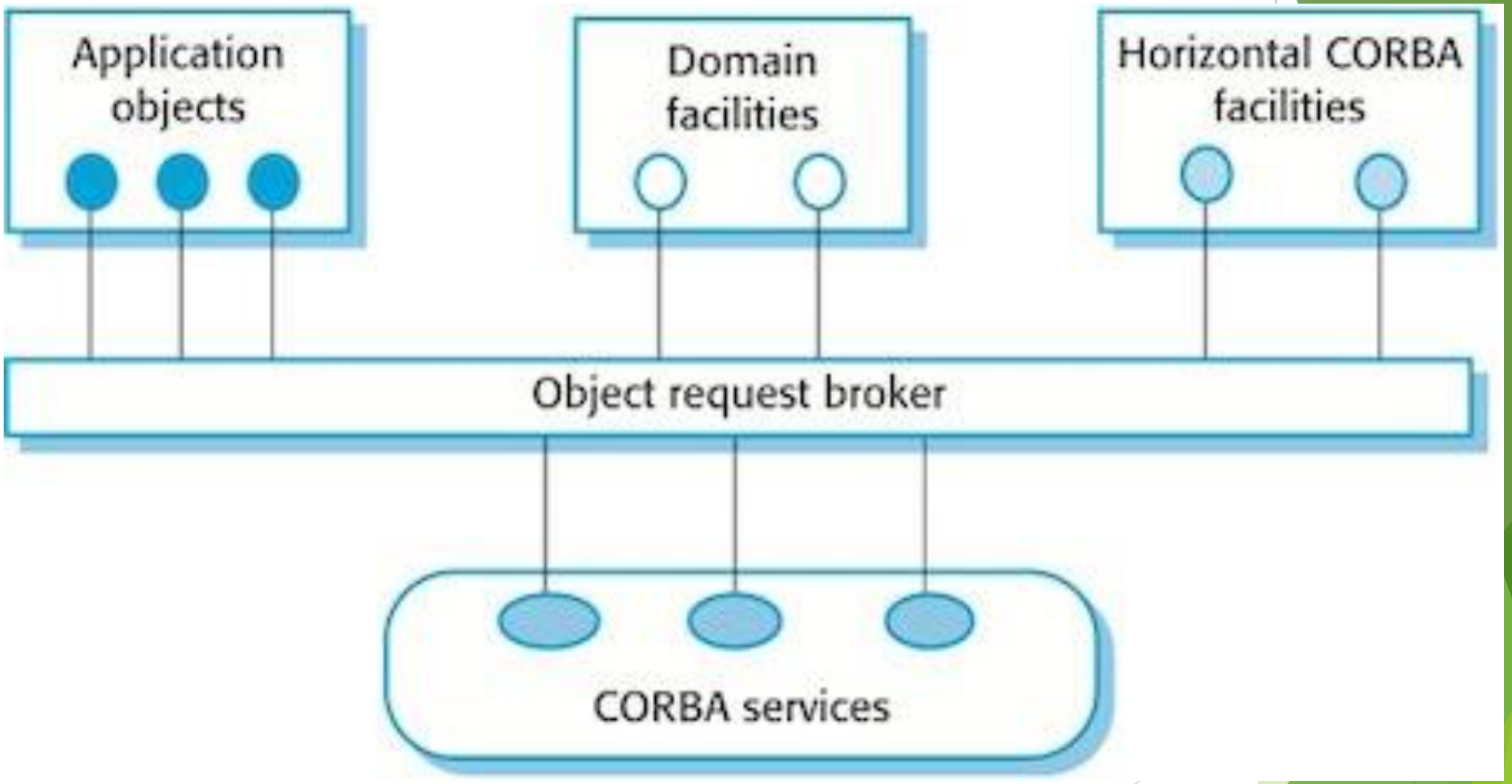
- Black-box frameworks promote maximum decoupling between the framework and its components, leading to increased flexibility and extensibility.
- For this reason, most large and/or highly-configurable systems are black-box frameworks.
- The disadvantage is that black-box systems can involve many abstract entities, making them more difficult to understand and program.

CORBA – Common Object Request Broker Architecture

- CORBA is a standard for an object request broker architecture that was developed by the Object Management Group in the 1990s. It was hoped that this would be widely adopted and that implementations of the standard would be available from competing vendors. The OMG standards are available from their website.
- The OMG's vision of a distributed application is shown in Figure 1, which I have adapted from Siegel's diagram of the Object Management Architecture (Siegal, 1998). This proposes that a distributed application should be made up of a number of components:

cont.:

- Application objects that are designed and implemented for this application.
- Standard objects that are defined by the OMG for a specific domain. These domain object standards cover finance/insurance, electronic commerce, healthcare, and a number of other areas.
- Fundamental CORBA services that provide basic distributed computing services such as directories, security management, etc.
- Horizontal CORBA facilities such as user interface facilities, system management facilities, etc. The term horizontal facilities suggests that these facilities are common to many application domains and the facilities are therefore used in many different applications.


Application objects
Domain facilities
Horizontal CORBA facilities
Object request broker
CORBA services

Cont.:

- The CORBA standards cover all aspects of this vision. There are four major elements to these standards:
- An object model for application objects where a CORBA object is an encapsulation of state with a well-defined, language-neutral interface described in an IDL (Interface Definition Language).
- An object request broker (ORB) that manages requests for object services. The ORB locates the object providing the service, prepares it for the request, sends the service request and returns the results to the requester.
- A set of object services that are general services likely to be required by many distributed applications. Examples of services are directory services, transaction services and persistence services.
- A set of common components built on top of these basic services that may be required by applications. These may be vertical domain-specific components or horizontal, general-purpose components that are used by many applications.

Cont.:

- The CORBA object model considers an object to be an encapsulation of attributes and services, as is normal for objects.
- However, CORBA objects must have a separate interface definition that defines the public attributes and operations of the object.
- CORBA object interfaces are defined using a standard, language-independent interface definition language (IDL).
- If an object wishes to use services provided by another object then it accesses these services through the IDL interface. CORBA objects have a unique identifier called an Interoperable Object Reference (IOR).
- This IOR is used when one object requests services from another. These requests are processed by the object request broker.

End

!!!!!!!